

## CSCE 411: Design and Analysis of Algorithms

### Week 4: Amortized Analysis

*Date: September 10, 2026*

*Nate Veldt, updated by Samson Zhou*

#### Course Logistics

- Amortized analysis: Chapter 17
- First test is next Tuesday

## 1 Simple Iterative Runtime Analysis

For simple iterative algorithms, a basic approach is to proving a runtime is to:

## 2 The Multi-pop Stack

Consider a stack  $S$  that has the following three operations

- $\text{PUSH}(S, v)$ : puts value  $v$  on the top of the stack
- $\text{POP}(S)$ : removes value on the top of the stack
- $\text{ISEMPTY}(S)$ : returns whether  $S$  is empty
- $\text{MULTIPOP}(S, k)$ :
  - Calls  $\text{POP}$
  - Pops
  - $k$  can be any integer number, different calls may involve different values of  $k$

**Question 1.** Assume that a single push or pop operation takes  $O(1)$  time. If we start with an empty stack, what is the worst-case runtime bound we obtain for applying  $n$  pop, push, and multipop operations, by applying a simple iterative runtime analysis?

- A**  $O(1)$
- B**  $O(n)$
- C**  $O(nk)$
- D**  $O(n^2)$

---

PUSHPOPFUN( $n$ )

Initialize empty multipop stack  $S$

**for**  $i = 1$  to  $n$  **do**

    Generate random number  $b \in [0, 1]$

**if**  $b < 1/3$  **then**

        PUSH( $S, 1$ )

**else if**  $b \in [1/3, 2/3)$  **then**

        POP( $S$ )

**else**

        Generate random integer  $k \in [1, n]$

        MULTIPOP( $S, k$ )

**end if**

**end for**

---

### 3 Amortized Analysis

In *amortized analysis*, we obtain improved runtime bounds for iterative methods by bounding the \_\_\_\_\_

#### 3.1 Technique 1: Aggregate Analysis

Let  $T(n)$  be:

\_\_\_\_\_  
Note that MULTIPOP( $S, k$ ) performs \_\_\_\_\_

The average cost per iteration (or *amortized cost*) is then

**Theorem 3.1.** PUSHPOPFUN( $n$ ) makes at most \_\_\_\_\_ individual push/pop operations.

**An important distinction** Our runtime analysis did not involve probabilistic reasoning. There are other notions of “average cost” for probabilistic algorithms, but we are not considering these. Our analysis holds independent of the random numbers generated in Algorithm PUSHPOPFUN.

### 3.2 Another approach for analyzing multipop stacks

If  $S$  is a multipop stack, and if it starts empty, then it is possible to pop an element  $v$  only if *we previously we pushed  $v$  onto  $S$* .

*Key idea:* Let's "pay" in advance for an eventual POP of that element.

- Whenever we push a new element, \_\_\_\_\_
- If/when that element is popped \_\_\_\_\_
- This is true whether we called \_\_\_\_\_
- In  $n$  iterations, there are at most  $n$  push operations and so \_\_\_\_\_

*Alternative view:* Each time we "overpay" for a push, we put money in the bank to pay for future pops.

### 3.3 Technique 2: The accounting method

Define:

- $c_i$  = the actual cost incurred at step  $i$
- $\hat{c}_i$  the \_\_\_\_\_

Here's how the accounting method works:

- The runtime we want to bound is:
- We choose convenient  $\hat{c}_i$  for each iteration so that

**Multipop Stack Example** In this example,

$c_i$  = number of pop/push operations in iteration  $i$ .

$$\hat{c}_i = \begin{cases} \text{_____} & \text{if we push in step } i \\ \text{_____} & \text{if we pop or multipop in step } i \end{cases}$$

### 3.4 Technique 3: The potential method

In the potential method, we identify some data structure and define:

- $c_i$ : the actual cost incurred at step  $i$
- $D_i$ : the state of a data structure after the  $i$ th iteration
- $\Phi$ : potential function defined on data structure;  $\Phi(D_i)$  is a real number
- $\hat{c}_i = \underline{\hspace{10em}}$  is the amortized cost

If  $\underline{\hspace{10em}}$ , we can bound actual cost in terms of amortized costs as follows:

**Multipop Stack Example** Define:

$c_i$  = number of push/pop operations at step  $i$

The data structure is \_\_\_\_\_

Define  $\Phi(D_i) =$  \_\_\_\_\_

Notice then that

$$\Phi(D_i) - \Phi(D_0) = \underline{\hspace{2cm}}$$

At step  $i$ , we “pay”  $\hat{c}_i = c_i + \Phi(D_i) - \Phi(D_{i-1})$

*In-Class Activity*

1. Write down the amortized cost  $\hat{c}_i$  in three different cases:
  - (a) In iteration  $i$ , a pop happens
  - (b) In iteration  $i$ , a push happens
  - (c) In iteration  $i$ , a multipop happens
2. Bound the runtime using step 1.

### Similarities and differences with the accounting method

- Similarity: both store “credit” and “pay” ahead of time
- The accounting method stores credit in individual steps
- The potential method stores credit as “potential” in a data structure
- For potential method, choose  $D_i$  and  $\Phi$  and prove  $\Phi(D_i) \geq \Phi(D_0)$ . \_\_\_\_\_
- For accounting, there is no  $D_i$  or  $\Phi$ . You must choose  $\hat{c}_i$  and prove \_\_\_\_\_
- For both, bound \_\_\_\_\_ to prove runtime guarantee.

## 4 Representing numbers in binary

We represent a number  $x \in \mathbb{N}$  in binary by a vector of bits  $A[0..k]$ , where  $A[i] \in \{0, 1\}$  and

$$x = \sum_{i=0}^k A[i] \cdot 2^i$$

## 5 The Binary Counter Problem

Let INCREMENT be an algorithm that takes in a binary vector and adds one to the binary number that it represents.

---

INCREMENT( $A$ )

$i = 0$

**while**  $i < A.length$  and  $A[i] == 1$  **do**

$A[i] = 0$

$i = i + 1$

**end while**

**if**  $i < A.length$  **then**

---

  
    **end if**

---

**Question 2.** If  $A$  is length  $k$  binary vector, what is the worst-case runtime for calling INCREMENT( $A$ )?

**A**  $O(1)$

**B**  $O(\log k)$

**C**  $O(k)$

**D**  $O(n)$

### 5.1 The actual cost of each iteration

Assume it takes  $O(1)$  time to check an entry of  $A$  or to flip its bit. At each step, the runtime is then just  $O(\text{number of flipped bits})$ , so we will say the cost of an iteration is

$c_i =$  \_\_\_\_\_

*Key idea:* Separate the costs that happen \_\_\_\_\_

|    |   |   |   |   |   |
|----|---|---|---|---|---|
| 0  | 0 | 0 | 0 | 0 | 0 |
| 1  | 0 | 0 | 0 | 0 | 1 |
| 2  | 0 | 0 | 0 | 1 | 0 |
| 3  | 0 | 0 | 0 | 1 | 1 |
| 4  | 0 | 0 | 1 | 0 | 0 |
| 5  | 0 | 0 | 1 | 0 | 1 |
| 6  | 0 | 0 | 1 | 1 | 0 |
| 7  | 0 | 0 | 1 | 1 | 1 |
| 8  | 0 | 1 | 0 | 0 | 0 |
| 9  | 0 | 1 | 0 | 0 | 1 |
| 10 | 0 | 1 | 0 | 1 | 0 |
| 11 | 0 | 1 | 0 | 1 | 1 |

## 5.2 Technique 1: Aggregate Analysis

Let  $T(n)$  be the total number of number of flipped bits in  $n$  increments.

**Question 3.** Look at the table on the last page, and observe how often the bit in position  $A[j]$  is flipping. If you wish, fill in the number of total flipped bits at each increment. What is the total cost of calling INCREMENT  $n$  times?

**A**  $O(n)$

**B**  $O(k)$

**C**  $O(nk)$

**D**  $O(n^2)$

### 5.3 Technique 2: The accounting method

- $c_i$  = the actual number of bits flipped
- $\hat{c}_i$  amortized cost for flipping bits: \_\_\_\_\_

|   |   |   |   |   |   |
|---|---|---|---|---|---|
| 0 | 0 | 0 | 0 | 0 | 0 |
| 1 | 0 | 0 | 0 | 0 | 1 |
| 2 | 0 | 0 | 0 | 1 | 0 |
| 3 | 0 | 0 | 0 | 1 | 1 |
| 4 | 0 | 0 | 1 | 0 | 0 |
| 5 | 0 | 0 | 1 | 0 | 1 |
| 6 | 0 | 0 | 1 | 1 | 0 |
| 7 | 0 | 0 | 1 | 1 | 1 |
| 8 | 0 | 1 | 0 | 0 | 0 |
| 9 | 0 | 1 | 0 | 0 | 1 |

---

INCREMENT( $A$ )

```

 $i = 0$ 
while  $i < A.length$  and  $A[i] == 1$  do
     $A[i] = 0$ 
     $i = i + 1$ 
end while
if  $i < A.length$  then
     $A[i] = 1$ 
end if

```

---

## 5.4 Technique 3: The potential method

**Step 0:** Identify data structure, potential function, and costs.

- $c_i$ : the actual number of flipped bits at iteration  $i$
- data structure: \_\_\_\_\_, with state  $D_i$  after iteration  $i$
- $\Phi(D_i) = b_i =$  \_\_\_\_\_
- $\hat{c}_i =$  \_\_\_\_\_

**Step 1:** Check that  $\Phi(D_i) \geq \Phi(D_0)$

**Step 2:** Compute and bound amortized costs

Let  $t_i$  be the number of bits that are set to 0 (*while* loop in algorithm) in iteration  $i$ .

If  $b_i = 0$ , this means we had  $A = [1\ 1\ 1\ \cdots\ 1]$  in iteration  $i$ , and incrementing turned it into  $A = [0\ 0\ 0\ \cdots\ 0]$ , meaning  $b_{i-1} = t_i = k$ .

**Question 4.** If  $b_i > 0$ , which of these is the right expression for  $b_i$ ?

**A**  $b_i = b_{i-1} + 1$

**B**  $b_i = t_i + 1$

**C**  $b_i = b_{i-1} + t_i$

**D**  $b_i = b_{i-1} - t_i$

**E**  $b_i = b_{i-1} + t_i + 1$

**F**  $b_i = b_{i-1} - t_i + 1$

Whether or not  $b_i = 0$ , we have a bound of \_\_\_\_\_

We can then compute the amortized cost and overall runtime bound: